Weekly Progress Report

Name: *Blake Simpson* **Week:** *3 (Baseline Prototype)* **Track:** *Track 3 — Agent*

WHAT I DID THIS WEEK

I built the first version of the agent that actually runs end-to-end. Weeks 1–2 only had the building blocks (a Remotive client, the Claude CLI wrapper, SQLite, a résumé loader). This week I wired them into a working agent and ran it on 5 test tasks, which is the Track 3 goal for the baseline (define tools → agent loop → run on 5 tasks).

Concretely I added:

- **A tools layer** (`src/job_scout/tools.py`) — the three actions the agent can take: `search_jobs` (Remotive), `score_jobs` (one batched Claude call that scores every posting 0–10 against the résumé), and `derive_query` (Claude turns the résumé into a search query). The two Claude tools each have a deterministic keyword fallback so the pipeline still runs if Claude is rate-limited.
- **The agent loop** (`src/job_scout/agent.py`) — search → score → decide → refine. It remembers which queries it has already tried, and if a round doesn't find enough good matches it derives a *different* query and goes again, up to a round limit. This is the actual "agent" part: memory + self-correction.
- **An entry point** (`src/job_scout/main.py`) so `job-scout` runs.
- **5 test tasks** — résumé fixtures in `tests/fixtures/` (Python backend, React frontend, ML engineer, DevOps/SRE, data analyst) and a runner (`scripts/run_baseline.py`) that runs the agent on all five and records the outputs to `docs/baseline_outputs.md` .

WHAT WORKED

- **It runs end-to-end on all 5 test tasks** against live Remotive + live Claude. Full outputs are in `docs/baseline_outputs.md` .

- **Claude scoring is good.** The 0–10 scores and one-line reasons are sensible — e.g. it correctly docks postings that are location-locked to Brazil or are QA/PM roles rather than the candidate's discipline. Batching all postings into a single Claude call (instead of one call per posting) made the run fast and cheap.
- **The agent loop behaves like an agent.** It derives a query from the résumé, and when round 1 finds nothing good it derives a second, distinct query and tries again — never repeating a query (the "tried queries" memory works).
- **The fallback path works.** Pointing `CLAUDE_PATH` at a missing binary makes the whole run fall back to keyword scoring and still complete — so "it runs" holds even with no Claude.

WHAT FAILED OR SURPRISED ME

- **The Remotive public API is currently ignoring my search query.** This was the big surprise. Every query — `python`, `react typescript frontend`, `data analyst sql`, even an empty query — returns the *same 33 postings* with `job-count: 33`. Adding `category=software-dev` changes nothing either. So the agent only ever has one tiny, fixed, mostly-irrelevant candidate pool to rank, which is why scores are low (mostly 1–5/10) and the *same* generic postings (a Brazil-only "Staff Product Engineer", several "Senior Quality Engineer" roles) top the list for *every* résumé. Remotive normally exposes thousands of jobs, so I'm likely being throttled/served a cached set, or their public endpoint has been reduced.
- **Importantly, the agent itself is not the problem here.** The scorer is doing the right thing — it gives low scores because the jobs genuinely don't match. The bottleneck is *search recall* (the data source), not scoring or the loop. Good thing to learn on a baseline week: my weak link is the input, not the model.
- No posting reached the 9/10 alert threshold in the *first* run — expected, given the candidate pool above. The threshold logic ran; it just had nothing to fire on.

HOW I RESOLVED THE REMOTIVE ISSUE

After the baseline I dug into *why* Remotive ignores the query. The `age` response header was **~38,000 seconds (~11 hours)** and every query returned the identical job ids — so it's a **stale CDN-cached response that never reaches Remotive's search backend.** Cache-busting (a unique `_=` param) didn't help; the CDN ignores unknown params. It's broken on their end, not mine — unfixable from the client.

So I stopped relying on Remotive's server-side search and made the search tool **multi-source with client-side filtering**:

- New `src/job_scout/remoteok.py` — pulls the RemoteOK public feed (~100 English tech jobs, no auth).
- New `src/job_scout/jobs.py` — fetches RemoteOK **and** Remotive once, merges and deduplicates them into a ~130-job corpus (cached per run), then **filters client-side by the query terms** (title matches weighted over tag/description). A dead source can't break the run; the others still supply candidates.
- `tools.search_jobs` now calls this aggregator instead of Remotive's search.

Result: different queries now return different, relevant jobs, and the score spread widened across the 5 tasks — the ML-engineer résumé surfaced a *9.0* "Senior Independent AI Engineer / Architect" (the alert threshold fired for the first time) and the React résumé surfaced a 7.0 "Frontend Developer". Updated outputs are in `docs/baseline_outputs.md`.

WHAT I LEARNED

- How to structure an agent as **tools + a loop with memory**, rather than one big prompt. Separating "search / score / derive query" into named tools made the loop readable and each piece independently testable.
- **Batch the LLM calls.** One Claude call scoring N postings is far cheaper and faster than N calls, and the model scores them more consistently side-by-side.
- Always give an LLM step a **deterministic fallback** for a baseline — it's the difference between "it runs" and "it runs only when the API is happy".
- Verify the *data source* the way I verified Claude. I'd confirmed Remotive was "reachable" in Week 2 but never confirmed that its `search` filter actually filters — and it doesn't right now.

EVIDENCE OF PROGRESS

- Baseline outputs for all 5 test tasks: `docs/baseline_outputs.md`.
- New code: `src/job_scout/tools.py`, `src/job_scout/agent.py`, `src/job_scout/main.py`, `src/job_scout/remoteok.py`, `src/job_scout/jobs.py`, `tests/fixtures/resume_*.md`, `scripts/run_baseline.py`.
- Run it: `python scripts/run_baseline.py` (or `job-scout <résumé>` for one).
- Repo: https://github.com/EXC3ll3NTrhyTHM/agent-workflow

Sample (ML-engineer test task, multi-source corpus + Claude scoring):

```
9.0  Senior Independent AI Engineer / Architect  [claude]  strong senior remote LLM/ML
3.0  Tech Lead Full-Stack Rails Engineer         [claude]  senior but Rails, not ML fo
1.0  Senior Product Manager                      [claude]  not an ML engineering role
tried: machine learning engineer llm, nlp rag engineer
```

PLAN FOR NEXT WEEK

- ~~Fix search recall~~ — **done.** Replaced Remotive's broken server-side search with a multi-source (RemoteOK + Remotive) corpus + client-side filtering. Next: add a third source (Arbeitnow or HN "Who is hiring") and tune the relevance filter.
- Build the **email-alert step** (Week 4 on the roadmap): wire Gmail SMTP so a ≥ threshold match sends a notification, using the existing `mark_alerted` dedup.
- Start the **score-stability check** I flagged in Week 2 — run the same résumé twice and measure how much a posting's score drifts, to decide whether the hard 9/10 threshold needs a multi-criteria rubric.

OPTIONAL: BLOCKERS / QUESTIONS FOR ZOOM

- Is it fine for the baseline that the *agent* works but the *data source* is currently degraded, or should I prioritise swapping job sources before anything else? My plan is to treat search recall as the Week 4 priority.